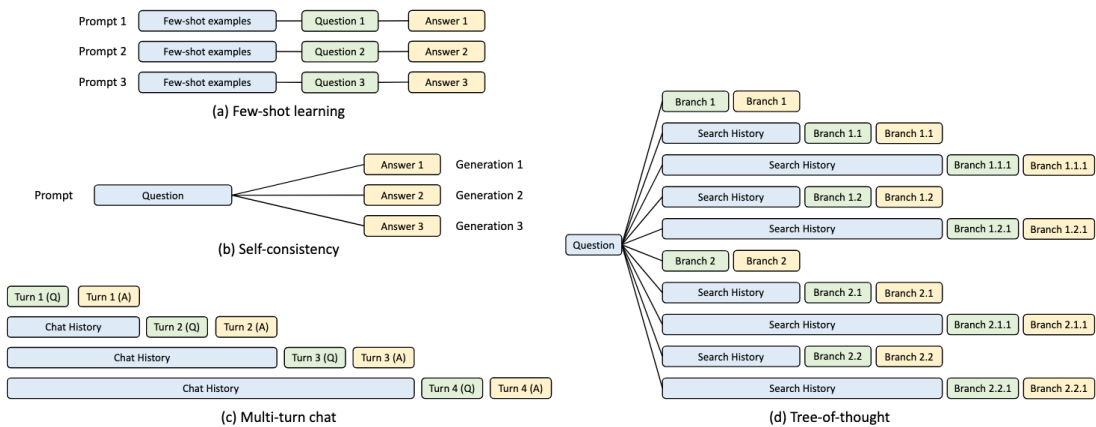


Prefix Caching 详解：实现 KV Cache 的跨请求高效复用

1 什么是 Prefix Caching

前缀缓存（Prefix Caching）是一种大语言模型推理优化技术，它的核心思想是缓存历史对话中的 KV Cache，以便后续请求能直接重用这些中间结果。这样可以显著降低首 token 延迟，提升整体推理效率。Prefix Caching 尤其适用于多轮对话、长文档问答等高前缀复用场景。

Prefix Caching 在大语言模型推理中的应用场景主要包括以下几类：



- **Few-shot learning (少样本学习)**：多个请求都包含相同的 few-shot 示例部分，只是最后的问题不同。Prefix Caching 可以将这些 few-shot 示例的 KV Cache 复用，避免每次都重新计算相同的示例内容。
- **Self-consistency (自洽性)**：对于同一个问题，先采样多个不同的推理路径（重复请求多次），然后选择最一致的答案。这些请求都共享相同的前缀（问题部分），Prefix Caching 可以让每次 decode 时都直接复用问题部分的缓存，只计算不同的答案部分。
- **Multi-turn chat (多轮对话)**：多轮对话中，每一轮的对话都基于之前的聊天历史。Prefix Caching 允许每一轮都复用之前聊天历史的KV缓存，只对新增的问答部分进行计算。
- **Tree-of-thought (思维树)**：复杂推理任务中，一个问题会被分解成多个分支，每个分支下又有进一步的分支。每个分支都共享前面的搜索历史作为前缀。Prefix Caching 可以让所有分支共享公共的历史部分缓存，只对各自独立的分支内容做增量计算。

Prefix Caching 只会减少处理查询（prefill 阶段）的时间，而不会减少生成新 token（decode 阶段）的时间。

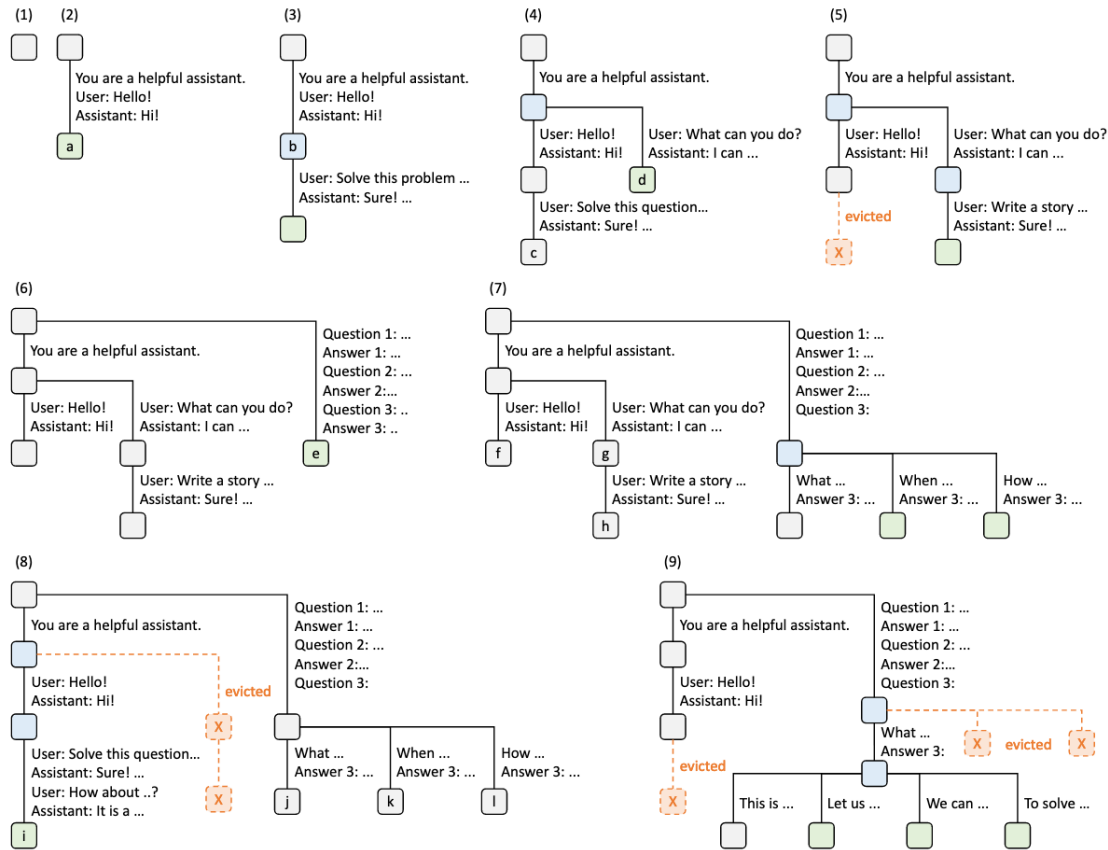
2 PagedAttention 和 Prefix Caching 的关系

- **PagedAttention** 主要解决 KV Cache 如何在 GPU 显存中“按需分配”，通过分页机制让 KV Cache 可以非连续存储和动态扩容，极大缓解内存碎片化问题，实现高效的内存管理。
- **Prefix Caching** 则专注于“避免重复算”，即当多个请求有相同的 prompt 前缀时，只需计算一次并缓存其 KV，后续请求直接复用，显著降低首 token 时延，尤其适合多轮对话和长 system prompt 场景。

维度	PagedAttention	Prefix Caching
关注点	高效管理 KV Cache 的内存分配与碎片化	复用请求间公共前缀的 KV Cache，减少重复计算
作用阶段	整个推理过程，包括 prefill 和 decode 阶段	prefill 阶段（推理开始前处理 prompt）
是否涉及跨请求	主要用于单个请求内部的缓存管理	针对不同请求间的共享前缀
技术原理	受操作系统虚拟内存分页启发，将 KV Cache 分块（block）动态分配和管理	通过哈希、基数树等结构检测和缓存相同前缀的 KV，跨请求复用
主要作用	解决 KV Cache 占用大、内存碎片严重、动态扩展难等问题，提升显存利用率和吞吐量	避免对相同前缀重复计算，显著降低首 token 延迟，提升多轮对话等场景效率
典型应用	任何高并发、长序列推理场景	长 system prompt、few-shot、对话历史复用、多轮对话等

3 RadixAttention

论文 [SGLang: Efficient Execution of Structured Language Model Programs](#) 中提出通过 RadixAttention 来实现 Prefix Caching。



上图展示了采用 LRU 淘汰策略的 RadixAttention 操作示例，描绘了 Radix Tree（基数树）在不同请求作用下的动态演化过程。这些请求包括两个对话会话、一批 few-shot 学习查询，以及一次自洽性采样（self-consistency sampling）。树的每条边标注了一个子字符串或一段 token 序列，节点则通过颜色编码以区分不同状态：

- **绿色**表示新添加的节点，
- **蓝色**表示当前时间点访问到的缓存节点，
- **红色**表示已经被淘汰的节点。

具体步骤如下：

1. **步骤(1)**: Radix Tree 初始为空。
2. **步骤(2)**: 服务器接收到用户消息 "Hello"，并生成 LLM 回复 "Hi"。系统提示 "You are a helpful assistant"、用户消息 "Hello!" 和模型回复 "Hi!" 被整合为一条边，并连接到一个新节点。
3. **步骤(3)**: 新的 prompt 到达，服务器在树中找到了该 prompt 的前缀（即第一轮对话），并重用其 KV cache。新的对话轮次作为新节点追加进树中。
4. **步骤(4)**: 开启新的对话会话。为了让两个会话共享系统提示，“b”节点被拆分成两个节点。
5. **步骤(5)**: 第二个会话继续，但由于内存限制，第(4)步中的“c”节点被淘汰。新的轮次被追加在“d”节点之后。
6. **步骤(6)**: 服务器收到一个 few-shot learning 查询，将其插入树中。由于该查询和现有节点没有公共前缀，根节点被拆分。
7. **步骤(7)**: 服务器收到一批新的 few-shot learning 查询。它们共享相同的 few-shot 示例，因此将(6)中的“e”节点拆分以实现共享。
8. **步骤(8)**: 服务器收到来自第一个对话会话的新消息。由于使用 LRU 策略，第二个对话的所有节点（如“g”和“h”）被淘汰。
9. **步骤(9)**: 服务器收到一个请求，要求对(8)中“j”节点的问题进行更多回答采样，可能是用于自治性采样（self-consistency sampling）。为了腾出空间，第(8)步中的“i”、“k”、“l”节点被淘汰。

4 vLLM 中的 Prefix Caching

最初，vLLM 支持手动前缀缓存，用户需通过 `prefix_pos` 参数显式指定前缀边界位置。

PR: <https://github.com/vllm-project/vllm/pull/1669>

从 [v0.4.0](#) 版本开始，vLLM 引入了**自动前缀缓存 (Automatic Prefix Caching)**，无需手动指定即可自动识别并复用共享前缀。

PR: <https://github.com/vllm-project/vllm/pull/2762>

4.1 在 vLLM 中启用 Prefix Caching

4.1.1 环境准备

执行以下命令安装 vLLM。

```
# 安装 uv，管理 python 虚拟环境
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env

# 安装 GPU Driver
wget https://cn.download.nvidia.com/tesla/565.57.01/NVIDIA-Linux-x86_64-565.57.01.run
sh NVIDIA-Linux-x86_64-565.57.01.run --silent

# 安装 CUDA Toolkit (如 nvcc、include、lib64)
```

```
sudo apt update
sudo apt install -y nvidia-cuda-toolkit

# 创建 python 虚拟环境
uv venv vllm-demo --python 3.12 --seed
source vllm-demo/bin/activate

# 安装 vLLM
uv pip install vllm
```

4.1.2 离线推理 (Offline Inference)

在 vLLM 中设置 `enable_prefix_caching=True` 可以启用 Automatic Prefix Caching。下面这段代码展示了 vLLM 的 Automatic Prefix Caching 功能：第一次生成关于 "John Doe 年龄" 的回答时，需要完整构建 KV Cache；而第二次询问 "Zack Blue 年龄"，由于两次问题共享相同的长表格前缀，vLLM 会自动复用已有缓存，从而显著减少重复计算，加速生成过程。

```
import time

from vllm import LLM, SamplingParams

LONG_PROMPT = (
    "You are a helpful assistant in recognizes the content of tables in markdown "
    "format. Here is a table as follows.\n# Table\n"
    + """
| ID | Name           | Age | Occupation   | Country      | Email
| Phone Number | Address
|-----|-----|-----|-----|-----|-----
| 1   | John Doe       | 29  | Engineer    | USA          |
john.doe@example.com | 555-1234      | 123 Elm St, Springfield, IL |
| 2   | Jane Smith     | 34  | Doctor      | Canada       |
jane.smith@example.com | 555-5678      | 456 Oak St, Toronto, ON   |
| 3   | Alice Johnson  | 27  | Teacher     | UK           |
| 555-8765      | 789 Pine St, London, UK   |
| 4   | Bob Brown      | 45  | Artist      | Australia    |
| 555-4321      | 321 Maple St, Sydney, NSW |
| 5   | Carol White    | 31  | Scientist   | New Zealand  |
| 555-6789      | 654 Birch St, Wellington, NZ |
| 6   | Dave Green     | 28  | Lawyer      | Ireland      |
| 555-3456      | 987 Cedar St, Dublin, IE  |
| 7   | Emma Black     | 40  | Musician    | USA          |
| 555-1111      | 246 Ash St, New York, NY  |
| 8   | Frank Blue     | 37  | Chef        | Canada       |
| 555-2222      | 135 Spruce St, Vancouver, BC |
| 9   | Grace Yellow   | 50  | Engineer    | UK           |
| 555-3333      | 864 Fir St, Manchester, UK |
| 10  | Henry Violet   | 32  | Artist      | Australia    |
| 555-4444      | 753 willow St, Melbourne, VIC|
| 11  | Irene Orange   | 26  | Scientist   | New Zealand  |
| 555-5555      | 912 Poplar St, Auckland, NZ |
| 12  | Jack Indigo    | 38  | Teacher     | Ireland      |
| 555-6666      | 159 Elm St, Cork, IE      |
| 13  | Karen Red      | 41  | Lawyer      | USA          |
| 555-7777      | 357 Cedar St, Boston, MA  |
    """
)
```

```

| 14 | Leo Brown      | 30 | Chef      | Canada      | leo.b@example.com
| 555-8888      | 246 Oak St, Calgary, AB      |
| 15 | Mia Green      | 33 | Musician   | UK           | mia.g@example.com
| 555-9999      | 975 Pine St, Edinburgh, UK   |
| 16 | Noah Yellow    | 29 | Doctor     | Australia    | noah.y@example.com
| 555-0000      | 864 Birch St, Brisbane, QLD   |
| 17 | Olivia Blue    | 35 | Engineer   | New Zealand  |
olivia.b@example.com | 555-1212      | 753 Maple St, Hamilton, NZ    |
| 18 | Peter Black    | 42 | Artist     | Ireland      | peter.b@example.com
| 555-3434      | 912 Fir St, Limerick, IE      |
| 19 | Quinn White    | 28 | Scientist  | USA          | quinn.w@example.com
| 555-5656      | 159 Willow St, Seattle, WA    |
| 20 | Rachel Red     | 31 | Teacher    | Canada       |
rachel.r@example.com | 555-7878      | 357 Poplar St, Ottawa, ON     |
| 21 | Steve Green    | 44 | Lawyer     | UK           | steve.g@example.com
| 555-9090      | 753 Elm St, Birmingham, UK    |
| 22 | Tina Blue     | 36 | Musician   | Australia    | tina.b@example.com
| 555-1213      | 864 Cedar St, Perth, WA       |
| 23 | Umar Black     | 39 | Chef       | New Zealand  | umar.b@example.com
| 555-3435      | 975 Spruce St, Christchurch, NZ|
| 24 | Victor Yellow  | 43 | Engineer   | Ireland      |
victor.y@example.com | 555-5657      | 246 Willow St, Galway, IE     |
| 25 | Wendy Orange   | 27 | Artist     | USA          | wendy.o@example.com
| 555-7879      | 135 Elm St, Denver, CO        |
| 26 | Xavier Green   | 34 | Scientist  | Canada       |
xavier.g@example.com | 555-9091      | 357 Oak St, Montreal, QC      |
| 27 | Yara Red       | 41 | Teacher    | UK           | yara.r@example.com
| 555-1214      | 975 Pine St, Leeds, UK        |
| 28 | Zack Blue     | 30 | Lawyer     | Australia    | zack.b@example.com
| 555-3436      | 135 Birch St, Adelaide, SA     |
| 29 | Amy White     | 33 | Musician   | New Zealand  | amy.w@example.com
| 555-5658      | 159 Maple St, Wellington, NZ   |
| 30 | Ben Black     | 38 | Chef       | Ireland      | ben.b@example.com
| 555-7870      | 246 Fir St, Waterford, IE      |
"""
)

```

```

def get_generation_time(llm, sampling_params, prompts):
    # time the generation
    start_time = time.time()
    output = llm.generate(prompts, sampling_params=sampling_params)
    end_time = time.time()
    # print the output and generation time
    print("-" * 30)
    print(f"Output: {output[0].outputs[0].text}")
    print(f"Generation time: {end_time - start_time} seconds.")
    print("-" * 30)

def main():
    # set enable_prefix_caching=True to enable APC
    llm = LLM(model="deepseek-ai/deepseek-llm-7b-chat",
enable_prefix_caching=True)

    sampling_params = SamplingParams(temperature=0, max_tokens=100)

```

```

# Querying the age of John Doe
get_generation_time(
    llm,
    sampling_params,
    LONG_PROMPT
    + "Question: what is the age of John Doe? Your answer: The age of John
Doe is ",
)

# Querying the age of Zack Blue
# This query will be faster since vllm avoids computing the KV cache of
LONG_PROMPT again.
get_generation_time(
    llm,
    sampling_params,
    LONG_PROMPT
    + "Question: what is the age of Zack Blue? Your answer: The age of Zack
Blue is ",
)

if __name__ == "__main__":
    main()

```

通过对比两次生成时间，发现第二次生成时间显著缩短，可以直观感受到 Automatic Prefix Caching 带来的性能提升。

```

-----
Output: 29.
Generation time: 0.46364879608154297 seconds.
-----
Adding requests: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████
| 1/1 [00:00<00:00, 180.41it/s]
Processed prompts: 100%|
████████████████████████████████████████████████████████████████████████████████
████████████████████████████████████████████████████████████████████████████████
| 1/1 [00:00<00:00, 7.95it/s, est.
speed input: 13891.30 toks/s, output: 31.84 toks/s]
-----
Output: 30.
Generation time: 0.13191604614257812 seconds.
-----

```

4.1.3 在线推理 (Online Serving)

在 GPU 后端中，v1 版本的 vLLM 默认启用 Prefix Caching (v0 默认禁用)，可以通过 `--no-enable-prefix-caching` 参数禁用 Prefix Caching。执行以下命令启动 vLLM 服务提供在线推理：

```
vllm serve deepseek-ai/deepseek-llm-7b-chat
```

然后使用以下 Python 代码请求在线推理服务，使用和前面离线推理相同的 prompt。

```
import time
```

```
import requests
```

```
LONG_PROMPT = (
```

```
    "You are a helpful assistant in recognizes the content of tables in markdown  
format. Here is a table as follows.\n# Table\n"
```

```
    + "```\n"
```

```
| ID | Name           | Age | Occupation   | Country       | Email  
| Phone Number | Address  
|-----|-----|-----|-----|-----|-----  
----|-----|-----|-----|-----|  
| 1 | John Doe      | 29 | Engineer    | USA           |  
john.doe@example.com | 555-1234 | 123 Elm St, Springfield, IL |  
| 2 | Jane Smith    | 34 | Doctor      | Canada        |  
jane.smith@example.com | 555-5678 | 456 Oak St, Toronto, ON |  
| 3 | Alice Johnson | 27 | Teacher     | UK            | alice.j@example.com  
| 555-8765 | 789 Pine St, London, UK |  
| 4 | Bob Brown     | 45 | Artist      | Australia     | bob.b@example.com  
| 555-4321 | 321 Maple St, Sydney, NSW |  
| 5 | Carol White   | 31 | Scientist   | New Zealand   | carol.w@example.com  
| 555-6789 | 654 Birch St, Wellington, NZ |  
| 6 | Dave Green    | 28 | Lawyer      | Ireland       | dave.g@example.com  
| 555-3456 | 987 Cedar St, Dublin, IE |  
| 7 | Emma Black    | 40 | Musician    | USA           | emma.b@example.com  
| 555-1111 | 246 Ash St, New York, NY |  
| 8 | Frank Blue    | 37 | Chef        | Canada        | frank.b@example.com  
| 555-2222 | 135 Spruce St, Vancouver, BC |  
| 9 | Grace Yellow  | 50 | Engineer    | UK            | grace.y@example.com  
| 555-3333 | 864 Fir St, Manchester, UK |  
| 10 | Henry Violet  | 32 | Artist      | Australia     | henry.v@example.com  
| 555-4444 | 753 Willow St, Melbourne, VIC|  
| 11 | Irene Orange  | 26 | Scientist   | New Zealand   | irene.o@example.com  
| 555-5555 | 912 Poplar St, Auckland, NZ |  
| 12 | Jack Indigo   | 38 | Teacher     | Ireland       | jack.i@example.com  
| 555-6666 | 159 Elm St, Cork, IE |  
| 13 | Karen Red     | 41 | Lawyer      | USA           | karen.r@example.com  
| 555-7777 | 357 Cedar St, Boston, MA |  
| 14 | Leo Brown     | 30 | Chef        | Canada        | leo.b@example.com  
| 555-8888 | 246 Oak St, Calgary, AB |  
| 15 | Mia Green     | 33 | Musician    | UK            | mia.g@example.com  
| 555-9999 | 975 Pine St, Edinburgh, UK |  
| 16 | Noah Yellow   | 29 | Doctor      | Australia     | noah.y@example.com  
| 555-0000 | 864 Birch St, Brisbane, QLD |  
| 17 | Olivia Blue   | 35 | Engineer    | New Zealand   |  
olivia.b@example.com | 555-1212 | 753 Maple St, Hamilton, NZ |  
| 18 | Peter Black   | 42 | Artist      | Ireland       | peter.b@example.com  
| 555-3434 | 912 Fir St, Limerick, IE |  
| 19 | Quinn White   | 28 | Scientist   | USA           | quinn.w@example.com  
| 555-5656 | 159 Willow St, Seattle, WA |  
| 20 | Rachel Red    | 31 | Teacher     | Canada        |  
rachel.r@example.com | 555-7878 | 357 Poplar St, Ottawa, ON |  
| 21 | Steve Green   | 44 | Lawyer      | UK            | steve.g@example.com  
| 555-9090 | 753 Elm St, Birmingham, UK |  
| 22 | Tina Blue     | 36 | Musician    | Australia     | tina.b@example.com  
| 555-1213 | 864 Cedar St, Perth, WA |  
| 23 | Umar Black    | 39 | Chef        | New Zealand   | umar.b@example.com  
| 555-3435 | 975 Spruce St, Christchurch, NZ|
```

```

| 24 | Victor Yellow | 43 | Engineer | Ireland |
victor.y@example.com | 555-5657 | 246 Willow St, Galway, IE |
| 25 | Wendy Orange | 27 | Artist | USA | wendy.o@example.com
| 555-7879 | 135 Elm St, Denver, CO |
| 26 | Xavier Green | 34 | Scientist | Canada |
xavier.g@example.com | 555-9091 | 357 Oak St, Montreal, QC |
| 27 | Yara Red | 41 | Teacher | UK | yara.r@example.com
| 555-1214 | 975 Pine St, Leeds, UK |
| 28 | Zack Blue | 30 | Lawyer | Australia | zack.b@example.com
| 555-3436 | 135 Birch St, Adelaide, SA |
| 29 | Amy White | 33 | Musician | New Zealand | amy.w@example.com
| 555-5658 | 159 Maple St, Wellington, NZ |
| 30 | Ben Black | 38 | Chef | Ireland | ben.b@example.com
| 555-7870 | 246 Fir St, Waterford, IE |
"""

```

```
)
```

```

def get_generation_time(prompt):
    url = "http://localhost:8000/v1/completions"
    headers = {"Content-Type": "application/json"}
    payload = {
        "model": "deepseek-ai/deepseek-llm-7b-chat",
        "prompt": prompt
    }

    start_time = time.time()
    response = requests.post(url, json=payload, headers=headers)
    end_time = time.time()

    print("-" * 30)
    if response.status_code == 200:
        result = response.json()
        output_text = result["choices"][0]["text"]
        print(f"Output: {output_text.strip()}")
        print(f"Generation time: {end_time - start_time} seconds.")
    else:
        print(f"❌ Error {response.status_code}: {response.text}")
    print("-" * 30)

def main():
    get_generation_time(
        LONG_PROMPT
        + "Question: what is the age of John Doe? Your answer: The age of John
Doe is "
    )

    get_generation_time(
        LONG_PROMPT
        + "Question: what is the age of Zack Blue? Your answer: The age of Zack
Blue is "
    )

if __name__ == "__main__":
    main()

```

输出结果如下：

```
Output: 29.
Generation time: 0.4827253818511963 seconds.
-----
Output: 30.
Generation time: 0.1334974765777588 seconds.
-----
```

4.2 实现原理

vLLM 选择了基于哈希的方法来实现 Prefix Caching。具体来说，vLLM 根据每个 KV block 内的 token 和该 block 之前前缀中的 token 来计算该 block 的哈希值：

```

          Block 1                Block 2                Block 3
    [A gentle breeze stirred] [the leaves as children] [laughed in the
distance]
Block 1: |<--- block tokens ---->|
Block 2: |<----- prefix ----->| |<--- block tokens ---->|
Block 3: |<----- prefix ----->| |<--- block tokens ---->|
```

在上面的示例中，第一个 block 的 KV Cache 可以通过 token “A gentle breeze stirred” 唯一标识。第三个 block 则可以通过 block 内的 token “laughed in the distance” 以及前缀 token “A gentle breeze stirred the leaves as children” 唯一标识。

此前，vLLM 中的每个序列都维护着一个从逻辑 KV block 到物理 KV block 的映射。为了实现 KV block 的自动缓存，vLLM 还将逻辑 KV block 映射到它们的哈希值，并维护一个全局哈希表用于管理所有物理 KV block。这样一来，所有具有相同哈希值的 KV block（例如不同请求之间共享的前缀 block）都可以映射到同一个物理 block，从而共享内存空间。这种设计实现了自动的前缀缓存，无需在 KV block 之间维护树状结构。

4.2.1 Block 的哈希值计算

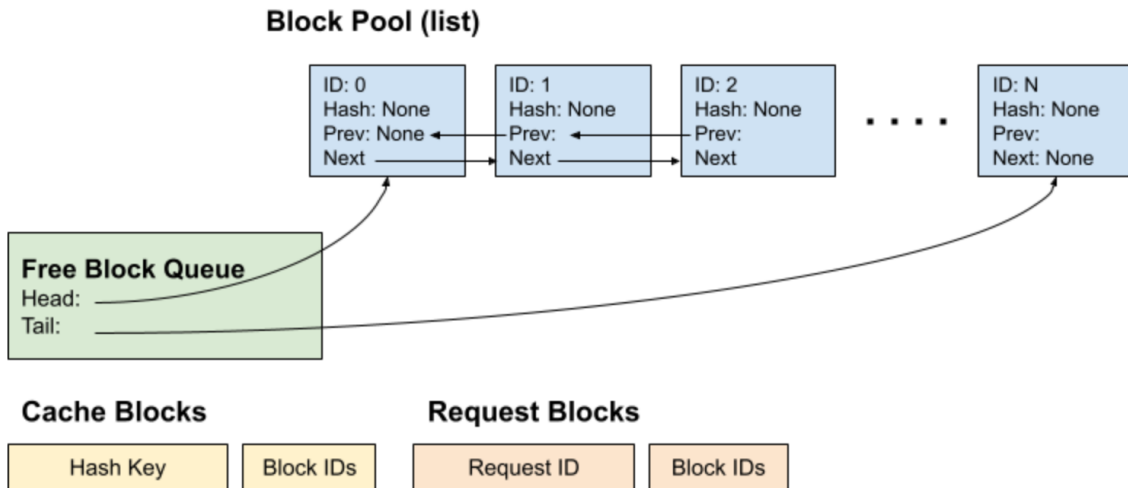
在 vllm v1 中，一个 block 的哈希值由 [3 个因素](#) 决定：

- parent_block_hash: 父 block 的哈希值。
- cur_block_token_ids: 该 block 中维护的 token ids。
- extra_keys: 用于确保该 block 唯一性的其他信息，例如 LoRA ID、多模态输入的哈希值，以及在多租户环境下用于隔离缓存的 cache salt 等。

```
BlockHashType(
    hash((parent_block_hash, curr_block_token_ids_tuple, extra_keys)),
    curr_block_token_ids_tuple,
    extra_keys
)
```

4.2.2 数据结构

在 vLLM 中实现 Prefix Caching 的数据结构如下图所示：



- [Block Pool](#): 管理所有 KV Cache block，提供分配、释放和缓存 block 的方法。Block Pool 包含所有的 `KVCacheBlock`，以及用于管理空闲块的 `FreeKVCacheBlockQueue`，同时还通过 [Cache blocks](#) (`cached_block_hash_to_block`) (`Dict[BlockHashType, Dict[block_id, KVCacheBlock]`) 维护哈希值与缓存 block 之间的映射关系。

```
class BlockPool:
    def __init__(self, num_gpu_blocks: int, enable_caching: bool):
        # All kv-cache blocks.
        self.blocks: list[KVCacheBlock] = [
            KVCacheBlock(idx) for idx in range(num_gpu_blocks)
        ]
        # Free block queue that constructs and manipulates a doubly linked
        # list of free blocks (including eviction candidates when caching is
        # enabled).
        self.free_block_queue = FreeKVCacheBlockQueue(self.blocks)

        # {block_hash: {block ID: block}}. A cached block is
        # a full block with a block hash that can be used for prefix caching.
        # The cached block may be used by running requests or in the
        # free_block_queue that could potentially be evicted.
        # NOTE: we currently don't de-duplicate the blocks in the cache,
        # meaning that if a block becomes full and is cached, we don't check
        # if there is already an identical block in the cache. This is because
        # we want to make sure the allocated block IDs won't change so that
        # block tables are append-only.
        self.cached_block_hash_to_block: dict[BlockHashType, dict[
            int, KVCacheBlock]] = defaultdict(dict)
```

- [Free Block Queue](#) ([free_block_queue](#) 属性, [FreeKVCacheBlockQueue](#) 实例)：是一个由 `KVCacheBlock` 组成的**双向链表结构**，用于维护所有空闲的 KV Cache block。队列本身仅维护 `head` 和 `tail` 指针，每个 block 通过其 `prev free block` 和 `next free block` 字段链接。该结构支持以 $O(1)$ 时间复杂度添加、删除或移动任意位置的 block，便于高效实现 LRU 淘汰策略和资源调度。

```

class FreeKVCacheBlockQueue:
    def __init__(self, blocks: list[KVCacheBlock]) -> None:
        self.num_free_blocks = len(blocks)

        # Initialize the doubly linked list of free blocks.
        self.free_list_head: Optional[KVCacheBlock] = blocks[0]
        self.free_list_tail: Optional[KVCacheBlock] = blocks[-1]

```

当一个 block 被分配后再释放时，会根据以下淘汰顺序重新添加到队列中（越靠前缓存越先被淘汰）：

1. **最近最少使用（LRU）的 block 排在最前；**
2. **如果多个 block 的最后访问时间相同**（例如由同一个请求分配），那么**哈希 token 数更多的 block 排在更前**。“哈希 token 数更多”在 vLLM 的中指的是在 block 链中位置更靠后的 block。在一个序列中：第一个块的哈希只依赖于其自身的 token，第二个块的哈希依赖于第一个块的哈希和自身的 token，第三个块的哈希依赖于第二个块的哈希和自身的 token，以此类推。因此序列末尾的块通常包含特定于当前请求的内容，复用价值较低序列开头的块（如系统提示）更可能在不同请求间共享。

- [Request blocks](#) 以及 [Block Pool](#) 都维护在 [KVCacheManager](#) 类中。
 - `req_to_blocks: Dict[req_id: List[KVCacheBlock]]`，记录一个请求下所有的 block。
 - `req_to_block_hashes: Dict[req_id, List[BlockHashType]]`，记录一个请求下所有的 block 的 hash 值。由于只有满块才可以被计算 hash 值，因此相同请求下，可能存在 `len(List[BlockHashType]) < len(List[KVCacheBlock])` 的情况。

```

class KVCacheManager:

    def __init__(
        self,
        kv_cache_config: KVCacheConfig,
        max_model_len: int,
        enable_caching: bool = True,
        caching_hash_algo: str = "builtin",
        num_preallocate_tokens: int = 64,
        log_stats: bool = False,
    ) -> None:
        self.block_pool = BlockPool(self.num_gpu_blocks, enable_caching)

        # Mapping from request ID to blocks to track the blocks allocated
        # for each request, so that we can free the blocks when the request
        # is finished.
        self.req_to_blocks: defaultdict[str,
                                         list[KVCacheBlock]] = defaultdict(list)

        # Mapping from request ID to kv block hashes.
        # This is to avoid recomputing the block hashes for each call of
        # `get_computed_blocks` or `allocate_slots`.
        self.req_to_block_hashes: defaultdict[
            str, list[BlockHashType]] = defaultdict(list)

        # {req_id: The number of cached blocks for this given request}
        # This is used to track the number of cached blocks for each request.
        # This is only used to track the RUNNING requests, we do not track the

```

```
# data for reemtped ones.  
self.num_cached_block: dict[str, int] = {}
```

4.2.3 操作

4.2.3.1 分配 Block

调度器为**新请求**分配 KV Cache block 的流程如下：

1. 调用 `kv_cache_manager.get_computed_blocks()`：
根据请求的 prompt tokens 进行哈希，并在缓存中查找对应的 Cache Blocks，获取已计算的 block 序列。
2. 调用 `kv_cache_manager.allocate_slots()`：执行以下步骤：
 - 计算当前请求需要分配的新 block 数量；若可用 block 数不足，则直接返回；
 - “触碰（touch）”已命中的缓存 block：即增加其引用计数，并将其从 Free Block Queue 中移除（如果当前没有其他请求在用），这样做是为了防止这些缓存 block 被淘汰。
 - 通过弹出 Free Block Queue 的队头来分配新 block；如果该 block 是缓存 block，则同时会驱逐该 block，其他请求将无法再复用此 block。
 - 如果新分配的 block 已经被 token 填满，则立即将其添加到 Cache Blocks 中，以便在同一批次中的其他请求可以复用。

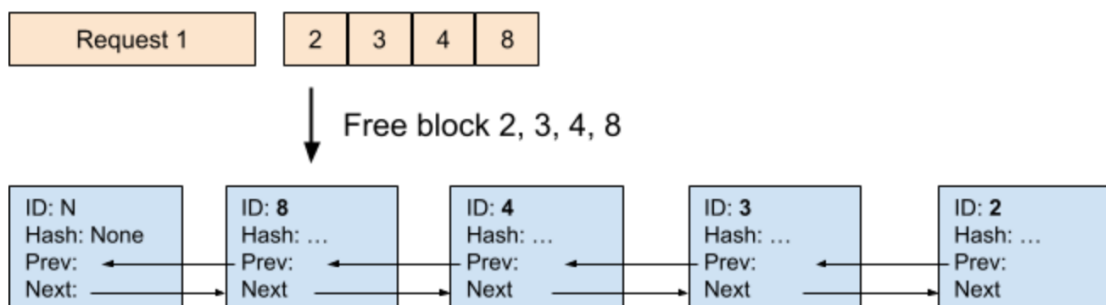
调度器为**运行中**的请求分配 KV Cache block 的流程如下：

调用 `kv_cache_manager.allocate_slots()`：执行以下步骤：

- 计算当前需要分配的新 block 数量；若可用 block 不足，则返回；
- 同样从 Free Block Queue 的队头弹出 block；如果弹出的 block 是缓存 block，则同时驱逐该 block，避免其他请求再复用；
- 将 token ID 写入已有 block 和新分配的 block 中的空槽位。如果某个 block 被填满，则将其添加到 Cache Blocks 中以进行缓存。

4.2.3.2 释放 Block

当一个请求结束时，如果其占用的 block 没有被其他请求使用（引用计数为 0），则释放这些 block。在本例中，释放了请求 1 以及其关联的 block 2、3、4 和 8。可以看到，释放的 blocks 会按照**逆序**添加到 Free Block Queue 的尾部。这是因为请求的最后一个 block 通常哈希了更多的 token，更具请求特异性，不太可能被其他请求复用，因此应当优先被淘汰。



4.2.3.3 驱逐 (LRU)

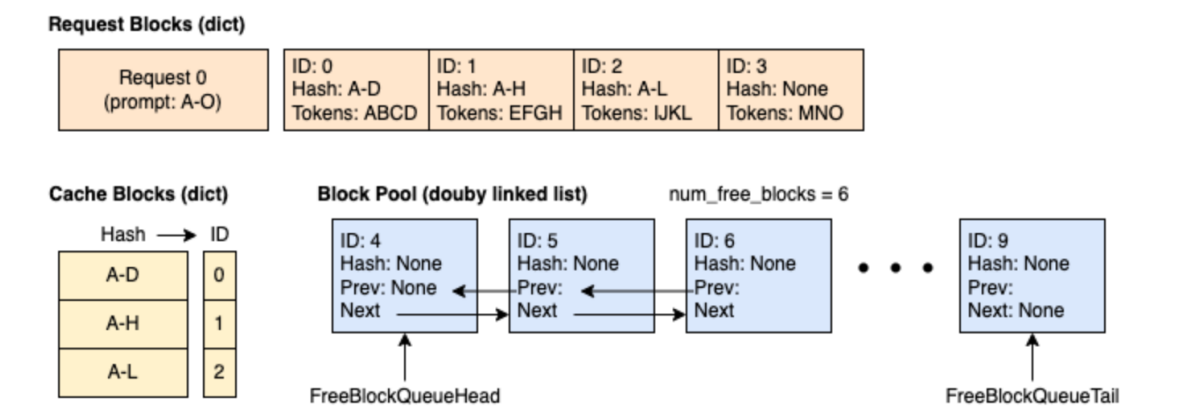
当 Free Block Queue 的队头 block（即最近最少使用的 block）仍处于缓存状态时，必须将其驱逐，以防止被其他请求继续使用。
具体的驱逐过程包括以下步骤：

- 从 Free Block Queue 的队头弹出该 block，即要被驱逐的 LRU block；
- 从 Cache Blocks 中移除该 block 的 ID；
- 从 KVCacheBlock 移除该 block 对应的哈希值。

4.3 示例

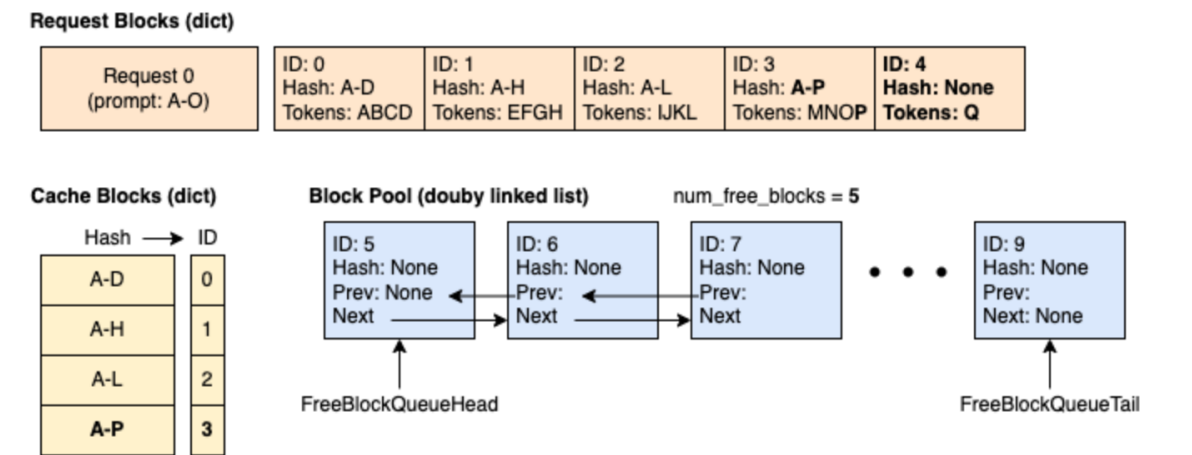
在本示例中，假设每个 block 的大小为 4（即每个 block 可缓存 4 个 token），整个 KV Cache Manager 中共有 10 个 block。

时刻 1：缓存为空，一个新请求 Request 0 (ABCD|EFGH|IJKL|MNO) 到来。分配了 4 个 block，其中 3 个已填满并被缓存，第 4 个 block 部分填充，仅包含 3 个 token。所有 prompt tokens 都被调度。



Block 的哈希值不是只基于自己的 token，而是包含了完整的前缀路径信息。例如，ID=2 的 hash 是“A-L”，表示这是一个对 token A 到 L 的 prefix 路径（前缀+当前块）的唯一哈希标识。

时刻 3：Request 0 经过 2 次推理过程（1 次 prefill + 1 次 decode），达到下面这个状态。Request 0 将 block 3 填满，并请求一个新 block 以继续 decode。此时将 block 3 缓存，并分配 block 4。



时刻 4：新的请求 Request 1 (ABCD|EFGH|IJKL|mn) 带着 14 个 prompt token 到来，其中前 10 个 token 与 Request 0 相同。可以看到，只有前两个 block（共 8 个 token）命中缓存，因为第 3 个 block 仅匹配了其 4 个 token 中的前 2 个。Request 1 使用的 block 5 已经被 token 填满，因此被缓存。

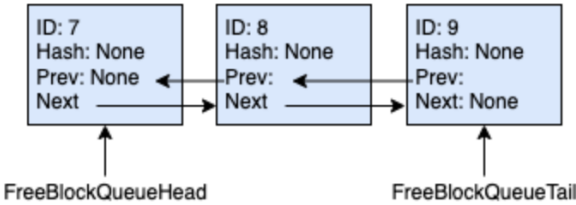
Request Blocks (dict)

Request 0 (prompt: A-O)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 2 Hash: A-L Tokens: IJKL	ID: 3 Hash: A-P Tokens: MNOP	ID: 4 Hash: None Tokens: QR
Request 1 (prompt: A-J,k-n)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 5 Hash: A-J,kl Tokens: IJkl	ID: 6 Hash: None Tokens: mn	

Cache Blocks (dict)

Hash	ID
A-D	0
A-H	1
A-L	2
A-P	3
A-J,kl	5

Block Pool (doubly linked list)



时刻 5: Request 0 已完成并被释放。Block 2、3 和 4 按照逆序被添加到空闲队列中（但 Block 2 和 3 仍处于缓存状态）。Block 0 和 1 未被加入空闲队列，因为它们仍被 Request 1 使用。

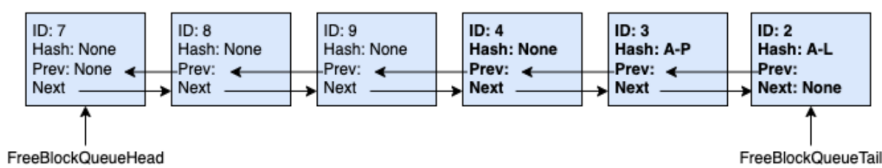
Request Blocks (dict)

Request 1 (prompt: A-J,k-n)	ID: 0 Hash: A-D Tokens: ABCD	ID: 1 Hash: A-H Tokens: EFGH	ID: 5 Hash: A-J,kl Tokens: IJkl	ID: 6 Hash: None Tokens: mno

Cache Blocks (dict)

Hash	ID
A-D	0
A-H	1
A-L	2
A-P	3
A-J,kl	5

Block Pool (doubly linked list)



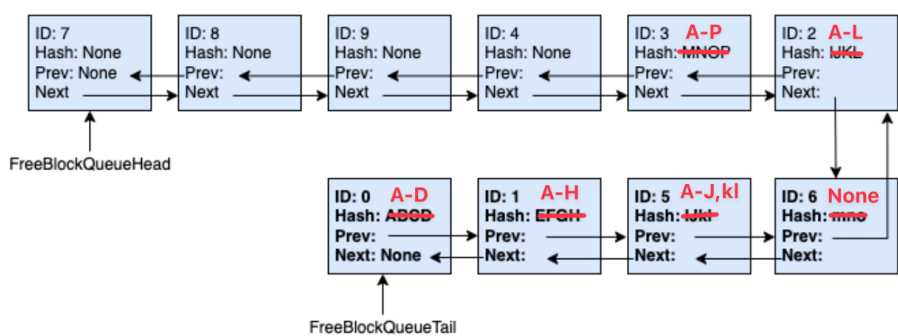
时刻 6: Request 1 推理完毕，同样需要释放掉相关资源。（原图有误，用红笔做了修正）

Request Blocks (dict)

Cache Blocks (dict)

Hash	ID
A-D	0
A-H	1
A-L	2
A-P	3
A-J,kl	5

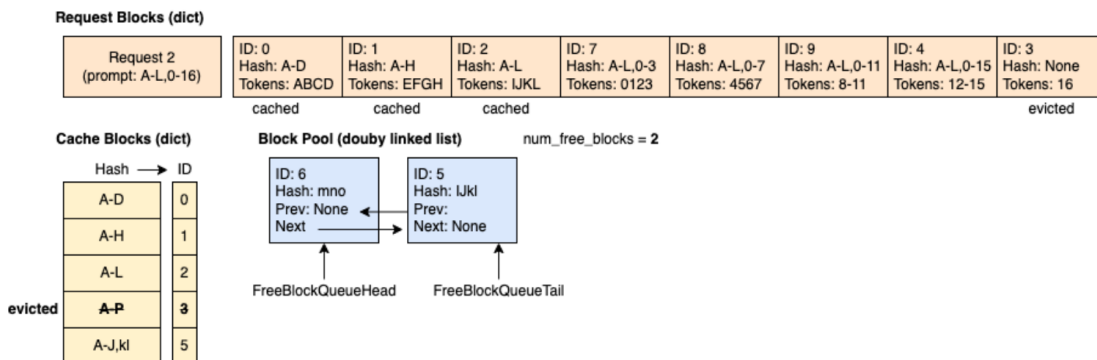
Block Pool (doubly linked list)



时刻 7: Request 2 (ABCD | EFGH | IJKL | 0-3 | 4-7 | 8-11 | 12-15 | 16) 带着 29 个 prompt token 到来，其中前 12 个 token 与 Request 0 完全相同。此时，前 3 个 block (block 0 ~ block 2) 可以命中缓存，因此在正式分配新 block 之前，会先被 touch 并从 Free Block Queue 中移除。队列顺序从原本的 7 - 8 - 9 - 4 - 3 - 2 - 6 - 5 - 1 - 0 更新为 7 - 8 - 9 - 4 - 3 -

6 - 5。剩余 5 个所需 block 将从 Free Block Queue 头部依次分配，因此获取了 block 7、8、9、4 和 3。由于 block 3 仍处于缓存状态（哈希值 A-P），因此需要将其从缓存中驱逐。

这个例子可以帮助我们更好体会到不立刻驱逐 block、以及逆序 append block 的好处。



4.4 几个注意点

4.4.1 只缓存完整的 block

在 vLLM 中只缓存完整的 block，假如一个 block 没有被 token 完全填满，那么这个 block 就不会被缓存。

```
# 假设 block_size = 4
# 请求的 token 序列如下：
tokens = ["A", "B", "C", "D", "E", "F", "G"]

# vLLM 会将 tokens 分成 KV blocks，每个 block 包含 4 个 token

# Block 0: ["A", "B", "C", "D"] ✓ - 完整的 block，满足 4 个 token，会被缓存
# Block 1: ["E", "F", "G"] ✗ - 只包含 3 个 token，未填满，不会被缓存
```

4.4.2 哈希冲突

哈希键结构并不能 100% 避免冲突。从理论上讲，不同的前缀 token 仍然有可能产生相同的哈希值。为了在多租户环境中避免哈希冲突，建议使用 SHA256 作为哈希函数，而不是默认的内置哈希。自 vLLM v0.8.3 起已支持 SHA256，可通过 `--prefix-caching-hash-algo` 命令行参数启用。但请注意，这会带来一定的性能开销：大约每个 token 增加 100-200 ns（对于 5 万个 token，大约增加 6 ms）。

4.4.3 前缀相同才能复用缓存

只有前缀相同的部分才能复用缓存，中间某一段相同是无法复用的。

```
假设对于 req1:
ABCD | EFGH

假设对于 req2:
DCAB | EFGH
```

虽然两者在 EFGH 部分的 token 内容完全一致，但 req2 不能复用 req1 的 EFGH block。这是因为 Transformer 的每一层都具有前向依赖性——每个 token 的表示不仅依赖它自身，还受到前面所有 token 的影响。因此，只要前缀不同，即使中间的 token 完全相同，其 KV 缓存结果也会不同，无法共享。

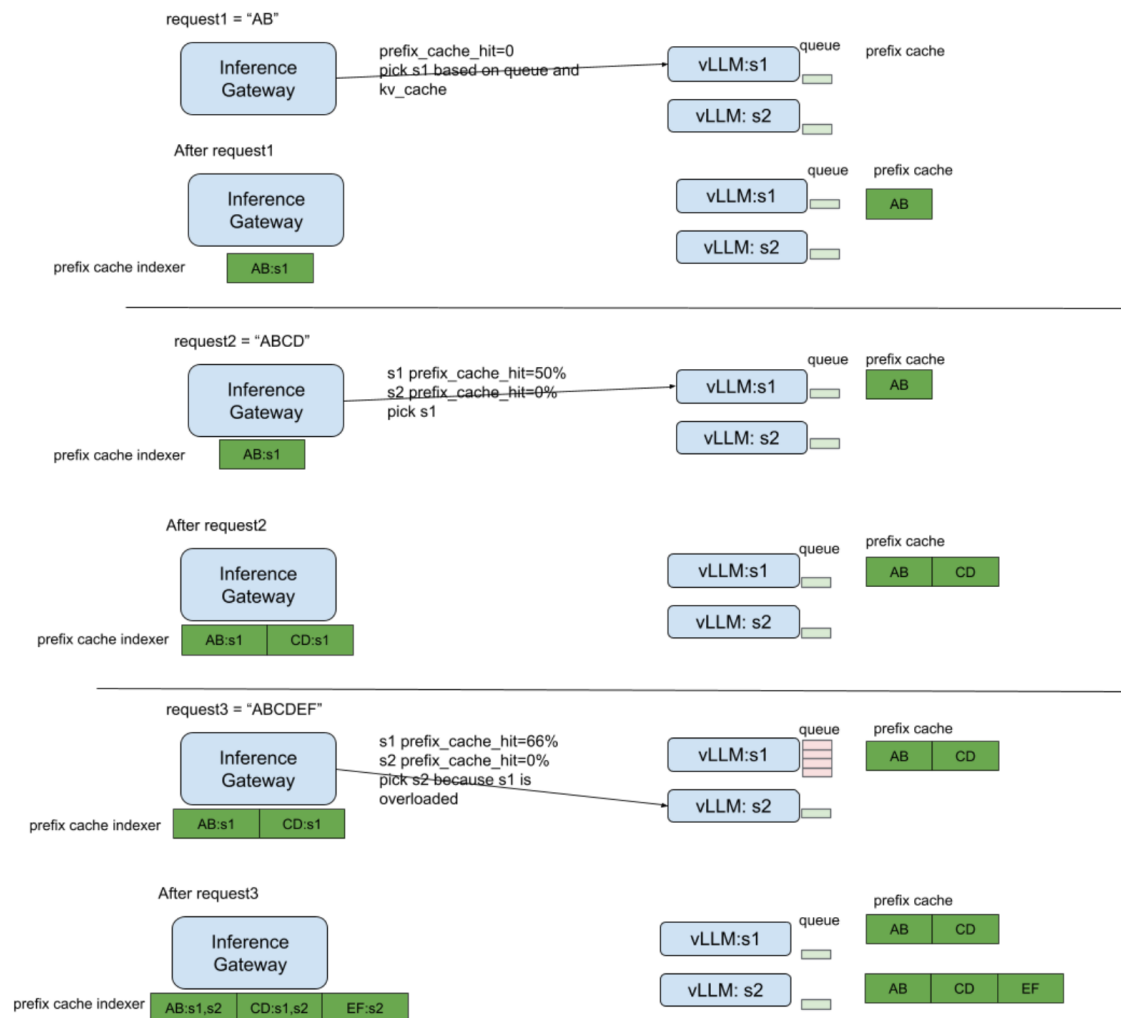
5 Prefix Cache Aware Routing

Prefix Caching 虽然能有效减少单个实例内部的 KV Cache 重复计算，但在多副本部署场景下，仅靠单实例的缓存复用远远不够。即使多个请求具有相同前缀，仍可能被随机分配到不同实例，导致每个实例都重复计算并缓存相同前缀。Prefix Cache Aware Routing 则是为了解决这个问题，它能根据请求前缀的匹配情况，智能地将请求路由到已有缓存的 worker，从而在集群层面实现更高效的 KV Cache 利用率。

目前，已经有不少项目实现了 Prefix Cache Aware Routing，例如：

- [vLLM Production Stack](#) 支持通过 [LMCache](#) 实现 Prefix Cache Aware Routing。另外 vLLM Production Stack 还有一个提案 [RFC: prefix-cache-aware routing](#) 中，其中实现了两种策略：基于 HashTrie 的匹配和基于 SimHash 的一致性哈希。其中，HashTrie 的方案在缓存命中率上表现更优。
- [SGLang](#) 则采用了一种基于请求历史构建 Radix Tree（基数树）的缓存感知路由策略。
- [AlBrix](#) 实现了一个分布式前缀缓存池，并对 vLLM 进行了定制化修改以支持从该缓存池加载缓存。在请求路由阶段，它的 [Prefix Router](#) 能最大化模型服务器上的前缀缓存命中率。目前支持两种策略：一种是类似 vLLM 的哈希匹配，另一种是类似 SGLang 的 Radix Tree 匹配。
- [KubeAI](#) 使用了一种带有负载边界的一致性哈希算法（CHWBL），它会对请求前缀（可配置长度）进行哈希，但可能因此牺牲一部分精度。当服务器负载过高时，它还会触发 "overflow" 策略将请求溢出到其他节点。
- [Gateway API Inference Extension](#) EPP ([End-point Picker](#)) 通过模拟模型服务器的缓存淘汰策略（如 LRU）构建一张所有后端服务器的近似前缀缓存索引表，用于指导后续请求的智能路由。关于 Gateway API Inference Extension 的详细解释可以参考：[为 Kubernetes 提供智能的 LLM 推理路由：Gateway API Inference Extension 深度解析](#)。

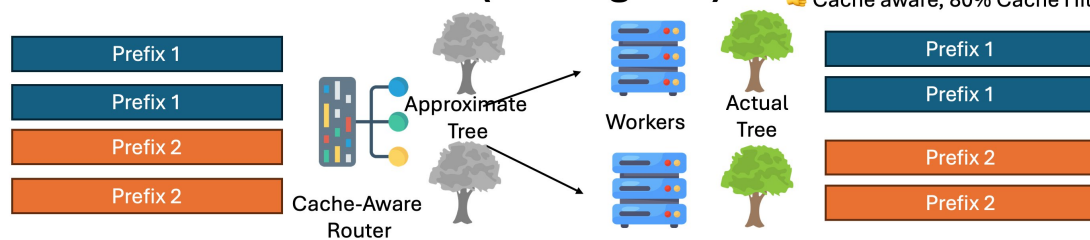
下图展示了 Gateway API Inference Extension 的 Prefix Cache Aware Routing 的工作流程。



SGLang v0.4 为 LLM 推理引擎引入了具备缓存感知（cache-aware）能力的负载均衡器。该负载均衡器能预测各个 worker 的 prefix KV cache 命中率，并自动选择匹配率最高的 worker。**测试显示其吞吐量最高提升 1.9 倍，缓存命中率改善达 3.8 倍**，且工作节点越多优势越显著。下图展示了缓存感知负载均衡器与传统轮询负载均衡器在数据并行中的差异。缓存感知负载均衡器会维护一个与 worker 实际基数树近似的基数树。该树会进行惰性更新，几乎没有任何开销。

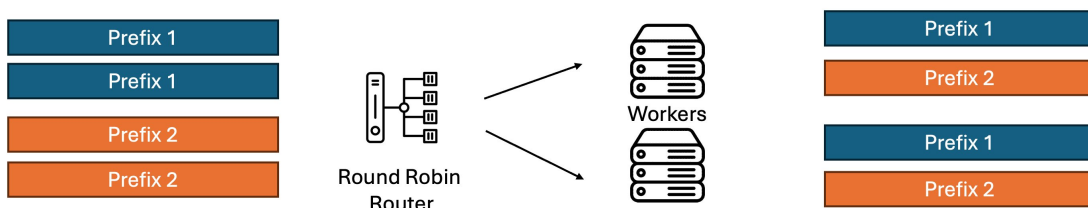
Cache Aware Data Parallel (SGLang v0.4)

👍 Cache aware, 80% Cache Hit Rate



Round Robin Data Parallel (SGLang v0.3)

👎 Not cache aware ~25% Cache Hit Rate



SGLang Router 的主要特性包括：

- **多节点支持**: 支持在多台机器上部署 worker, 单个 Router 可连接分布式的多个 worker, 便于水平扩展, 同时在分布式环境中保持对缓存命中的感知能力。
- **感知缓存的路由机制**: 将请求优先发送到缓存命中率更高的 worker, [并结合负载均衡策略避免负载不均](#)。
- **免通信设计**: worker 之间无需同步缓存状态, [Router 通过跟踪请求历史来近似推断各个 worker 的缓存状态](#), 而不是直接查询 worker 的实际缓存信息。
- **高性能实现**: 使用纯 Rust 编写, 支持高并发, 开销极低, 性能相比基于 Python 的方案提升达 2 倍。
- **独立包形式发布**: 以 `sglang-router` 包发布, 提供 Python 接口, 并配有 CLI 工具, 方便用户快速上手使用。

SGLang Router 在分布式系统层面优化多 worker 环境中的缓存利用率, 而核心的 prefix caching 则专注于单个 worker 内的计算重用。

使用方式如下, 先安装 `sglang` 和 `sglang-router` 包。

```
uv venv sglang-demo --python 3.12 --seed
source sglang-demo/bin/activate
uv pip install sglang[all]
uv pip install sglang-router
```

可以使用 `sglang_router.launch_server` 一起启动 SGLang Router 和多个 worker。 `--dp-size` 表示你要启动多少个独立的 worker 来进行数据并行 (data parallel)。这里启动了 2 个 worker, 因此你的服务器上需要 2 个 GPU。

```
python -m sglang_router.launch_server \
--model-path deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B \
--dp-size 2 --host 0.0.0.0
```

如果是在多个节点上启动 worker, 然后在主节点上启用 SGLang Router, 可以使用 `sglang_router.launch_router`。

```
# 先分别启动几个 worker
# 在第一个窗口执行
python -m sglang.launch_server --model-path deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B --host 0.0.0.0 --port 30001 --base-gpu-id 0
# 在第二个窗口执行
python -m sglang.launch_server --model-path deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B --host 0.0.0.0 --port 30002 --base-gpu-id 1

# 启动 SGLang Router
# 在第三个窗口执行
python -m sglang_router.launch_router \
--worker-urls http://localhost:30001 http://localhost:30002
```

再开启一个窗口发送请求到 SGLang Router, 反复发送多次请求:

```
curl -X POST http://localhost:30000/generate \
-H "Content-Type: application/json" \
-d '{"text": "what is the capital of France?"}'
```

可以看到请求始终落到其中一个 worker 上。（只会在一个 worker 的日志中看到请求信息）

```
[2025-06-08 21:06:35] Prefill batch. #new-seq: 1, #new-token: 7, #cached-token: 1, token usage: 0.00, #running-req: 0, #queue-req: 0
2025-06-08 21:06:35,733 - INFO - flashinfer.jit: Loading JIT ops: cascade
2025-06-08 21:06:35,741 - INFO - flashinfer.jit: Finished loading JIT ops: cascade
[2025-06-08 21:06:36] Decode batch. #running-req: 1, #token: 41, token usage: 0.00, cuda graph: True, gen throughput (token/s): 0.88, #queue-req: 0
[2025-06-08 21:06:36] Decode batch. #running-req: 1, #token: 81, token usage: 0.00, cuda graph: True, gen throughput (token/s): 122.53, #queue-req: 0
[2025-06-08 21:06:36] Decode batch. #running-req: 1, #token: 121, token usage: 0.00, cuda graph: True, gen throughput (token/s): 121.24, #queue-req: 0
[2025-06-08 21:06:36] INFO: 127.0.0.1:50554 - "POST /generate HTTP/1.1" 200 OK
[2025-06-08 21:06:38] Prefill batch. #new-seq: 1, #new-token: 1, #cached-token: 7, token usage: 0.00, #running-req: 0, #queue-req: 0
[2025-06-08 21:06:39] Decode batch. #running-req: 1, #token: 33, token usage: 0.00, cuda graph: True, gen throughput (token/s): 16.84, #queue-req: 0
[2025-06-08 21:06:39] Decode batch. #running-req: 1, #token: 73, token usage: 0.00, cuda graph: True, gen throughput (token/s): 122.95, #queue-req: 0
[2025-06-08 21:06:39] Decode batch. #running-req: 1, #token: 113, token usage: 0.00, cuda graph: True, gen throughput (token/s): 122.47, #queue-req: 0
[2025-06-08 21:06:39] INFO: 127.0.0.1:50554 - "POST /generate HTTP/1.1" 200 OK
[2025-06-08 21:06:41] Prefill batch. #new-seq: 1, #new-token: 1, #cached-token: 7, token usage: 0.00, #running-req: 0, #queue-req: 0
[2025-06-08 21:06:41] Decode batch. #running-req: 1, #token: 25, token usage: 0.00, cuda graph: True, gen throughput (token/s): 21.48, #queue-req: 0
[2025-06-08 21:06:41] INFO: 127.0.0.1:50554 - "POST /generate HTTP/1.1" 200 OK
```

在 SGLang Router 的日志上也可以看出，请求被转发给了 worker 1。

```
[Router (Rust)] 2025-06-08 21:06:08 - INFO - 🛠️ Initializing router on 127.0.0.1:30000
[Router (Rust)] 2025-06-08 21:06:08 - INFO - 🛠️ Initializing workers on ["http://localhost:30001", "http://localhost:30002"]
[Router (Rust)] 2025-06-08 21:06:08 - INFO - 🛠️ Policy Config: CacheAwareConfig { cache_threshold: 0.5, balance_abs_threshold: 32, balance_rel_threshold: 1.0001, eviction_interval_secs: 60, max_tree_size: 16777216, timeout_secs: 300, interval_secs: 10 }
[Router (Rust)] 2025-06-08 21:06:08 - INFO - 🛠️ Max payload size: 4 MB
[Router (Rust)] 2025-06-08 21:06:08 - INFO - All workers are healthy
[Router (Rust)] 2025-06-08 21:06:08 - INFO - ✅ Serving router on 127.0.0.1:30000
[Router (Rust)] 2025-06-08 21:06:08 - INFO - ✅ Serving workers on ["http://localhost:30001", "http://localhost:30002"]
[Router (Rust)] 2025-06-08 21:06:08 - INFO - starting 32 workers
[Router (Rust)] 2025-06-08 21:06:08 - INFO - Actix runtime found; starting in Actix runtime
[Router (Rust)] 2025-06-08 21:06:08 - INFO - starting service: "actix-web-service-127.0.0.1:30000", workers: 32, listening on: 127.0.0.1:30000
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Before eviction - Used size per tenant:
```

```
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Tenant: http://localhost:30001, Size: 0
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Tenant: http://localhost:30002, Size: 0
[Router (Rust)] 2025-06-08 21:07:08 - INFO - After eviction - Used size per tenant:
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Tenant: http://localhost:30001, Size: 0
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Tenant: http://localhost:30002, Size: 0
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Processed Queue: {"http://localhost:30002": 0, "http://localhost:30001": 3}
[Router (Rust)] 2025-06-08 21:07:08 - INFO - Running Queue: {"http://localhost:30002": 0, "http://localhost:30001": 0}
```

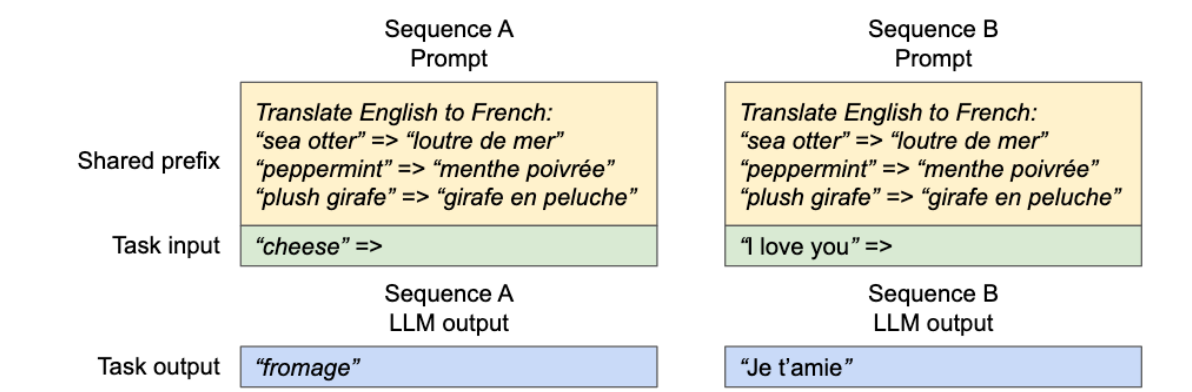
6 总结

Prefix Caching 通过缓存并复用多个请求中相同前缀的 KV Cache，有效降低了大语言模型推理中的首 token 延迟和计算成本。与 PagedAttention 关注内存管理不同，Prefix Caching 专注于跨请求的计算复用，特别适用于多轮对话、few-shot 学习等场景。实现方式上，SGLang 采用基数树 (RadixAttention) 方案，而 vLLM 则使用基于哈希的方法。在分布式部署环境中，Prefix Cache Aware Routing 进一步优化了集群级别的缓存利用率，通过智能路由将请求发送到缓存命中率更高的节点。

7 附录

7.1 Few-shot learning

Few-shot learning 就是通过在 prompt 中给模型少量任务示例，让模型在没有专门微调的情况下，理解并完成新任务。



7.2 Self-consistency

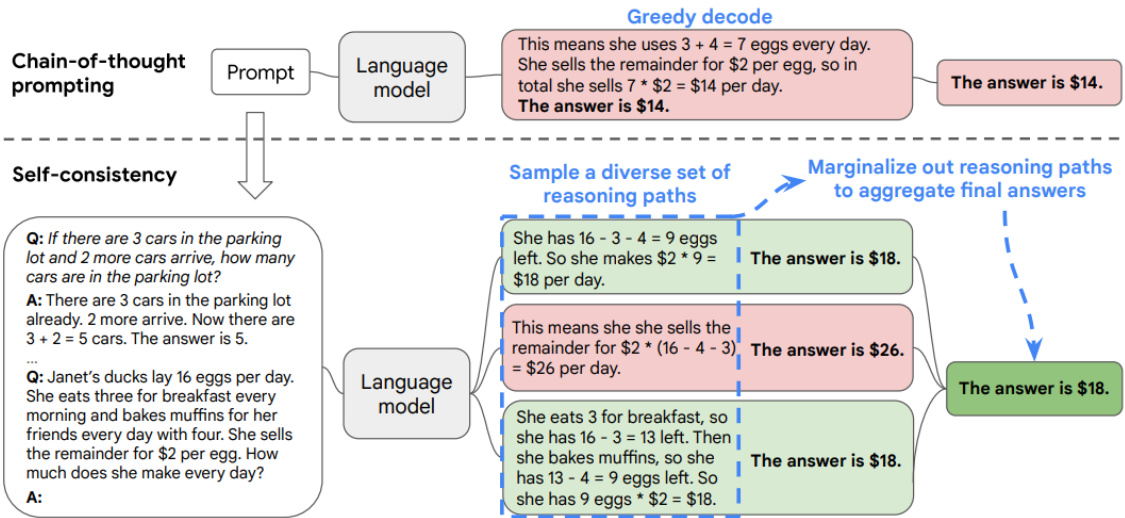
Self-consistency 的概念来源于论文 [Self-Consistency Improves Chain of Thought Reasoning in Language Models](#)。

该方法基于这样的假设：在复杂推理任务中，从问题到唯一正确答案通常存在多种不同的推理路径。其核心方案是用 **self-consistency 解码策略** 替代传统的贪婪解码。具体做法是：对语言模型进行多次采样，生成多条不同的推理路径（即重复请求多次），然后根据这些路径的最终答案进行投票，选出最一致的答案作为最终输出。

Self-consistency 策略认为复杂推理任务往往可以通过多条路径获得正确解，因此通过抽样生成一个多样化的推理路径集合，并选取一致性最高的结果，有效降低了贪婪解码带来的随机性。

Self-consistency 的核心流程如下：

- 1. **Step 1**：使用思维链（Chain-of-Thought）提示，引导模型进行逐步推理；
- 2. **Step 2**：对语言模型进行多次采样，生成多个推理路径；
- 3. **Step 3**：对不同路径的最终答案进行投票，选择一致性最高的答案输出。



7.3 Chain of Thought

Chain of Thought (CoT) 是一种增强语言模型推理能力的技术，特别适用于需要多步推理的问题。通过在模型的提示中加入一系列的中间推理步骤，可以帮助模型进行复杂的推理任务，从而避免单纯的“直接回答”模式。CoT 使得模型能够理解并生成推理过程，而不是直接给出答案，从而提高其在复杂问题上的表现。

CoT 有两种应用模式：

Few-Shot CoT

在 Few-Shot CoT 中，开发者给出一两个示例，在示例中明确展示如何进行思维链的推理。通过这些示例，模型能够学习如何通过逐步推理得出结论。

示例：

假设用户询问：“我想为朋友的生日挑选一束花。”

步骤1：理解问题，确定用户的需求。

步骤2：列出可能适合生日的花种。

步骤3：根据花的象征意义、花朵的颜色和花期，筛选出推荐的花种。

这种逐步思考的过程可以让模型根据需求生成符合用户期望的推荐。

Zero-Shot CoT

在 Zero-Shot CoT 中，开发者直接告诉模型进行逐步推理。例如，通过提示“让我们一步步地思考”，模型就能自动产生更清晰、合理的推理步骤，而不需要提前给出示例。

示例：

假设用户询问：“我想为我的女朋友购买一些花，她喜欢粉色和紫色的花。”
通过简单的提示：“让我们一步步思考”

模型就能给出以下推理过程：

步骤1：理解需求（粉色和紫色的花）。

步骤2：列举适合的花种（例如粉色的玫瑰、紫色的兰花等）。

步骤3：结合花的象征意义和花卉的实际情况（如价格、季节性等），给出推荐。

7.4 Tree of Thought

Tree of Thought (ToT) 进一步扩展了 CoT 的理念，特别适用于需要多步骤推理的复杂任务。与 CoT 不同，ToT 框架不仅要求生成思维链，而是生成多个思维路径，并通过“思维树”进行探索。每个思维步骤都具有多个备选方案，模型会在这些方案中搜索最优解。

示例：

假设用户询问：“我想为我的妻子买一束鲜花，但我不确定选择哪种。她喜欢淡雅的颜色和花香。”
在 ToT 框架下，模型会按照以下步骤进行思考：

思维步骤1：理解需求（淡雅的颜色和花香）。

思维步骤2：列出候选花种：百合、玫瑰、紫罗兰、桔梗、康乃馨。

思维步骤3：评估每种花是否符合要求（花香、颜色、花期等）。

思维步骤4：通过多条思维路径筛选出最优选择（如百合、紫罗兰等）。

最终推荐：基于推理过程给出具体建议，例如：“考虑到您妻子喜欢淡雅的颜色和花香，我建议选择百合或紫罗兰，它们既符合颜色要求又有花香。”

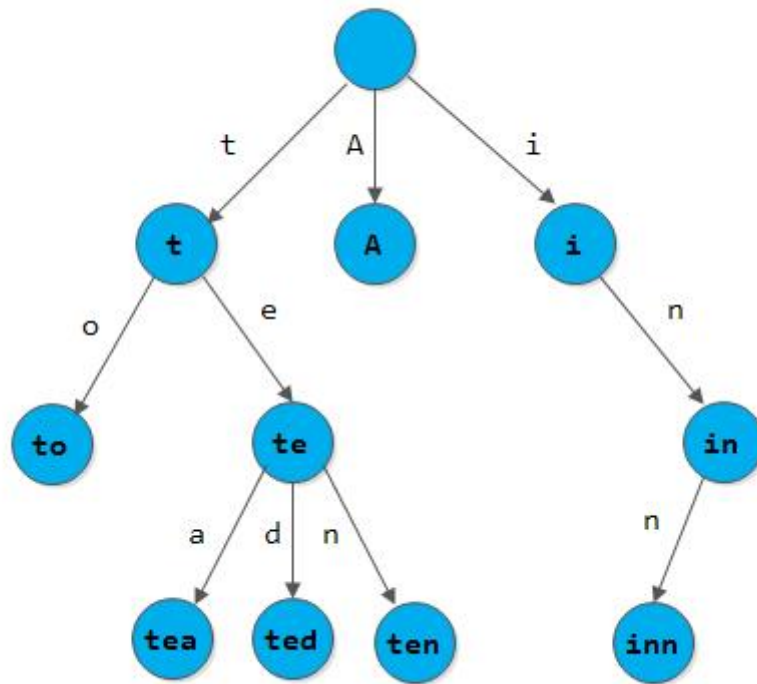
CoT 与 ToT 的区别与联系

- CoT：专注于引导模型逐步推理，强调思考的过程，可以通过单一路径进行推理并得出答案。
- ToT：在 CoT 的基础上，加入了多条推理路径的选择，使得模型能够在多条思维路径中搜索最优解。ToT 更适合处理复杂问题，尤其是需要多个选择和深度探索的场景。

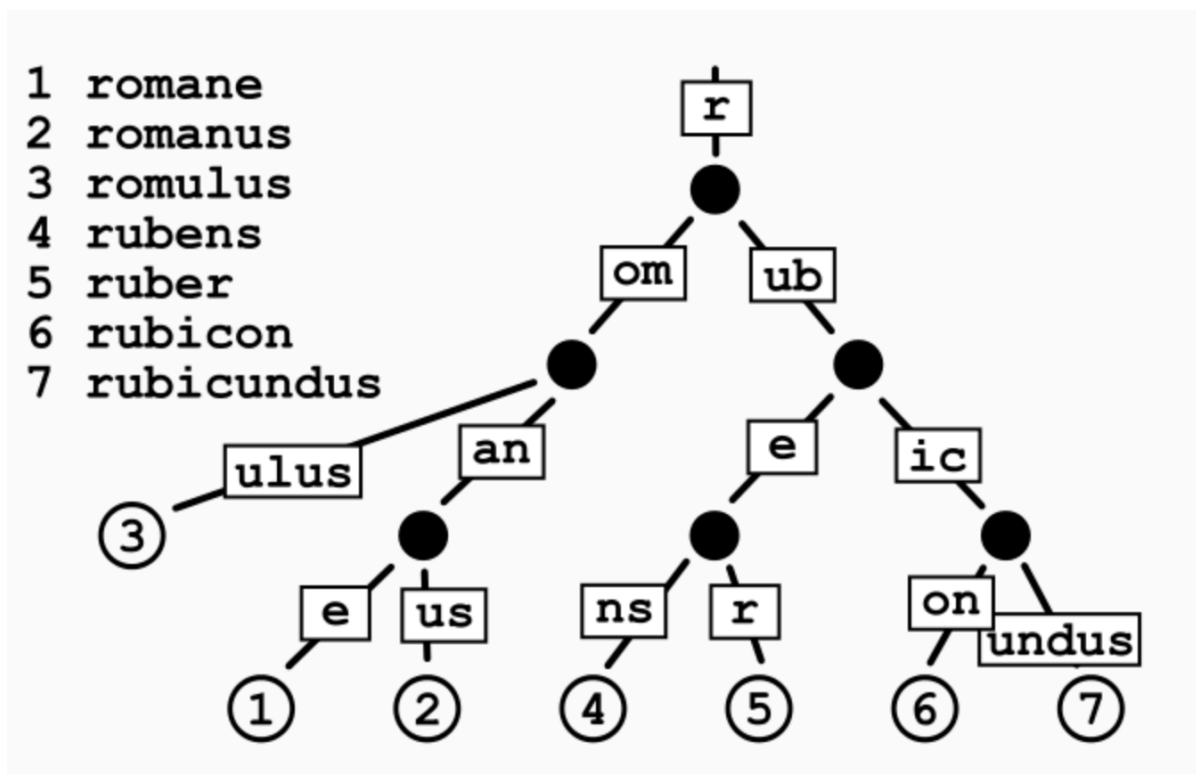
7.5 前缀树 (Trie) 和 基数树 (Radix Tree)

基数树 (Radix Tree) 和前缀树 (Trie) 的区别主要在于结构的紧凑性和节点的表示方式：

- **前缀树 (Trie)** 是一种按字符逐层拆分的树结构，每个节点只存储一个字符，路径上的字符连接起来表示字符串。它的层级深度通常等于字符串的长度，节点的子节点数较多（比如 26 个英文字母），空间利用率较低，但查找操作简单直观。Trie 这个术语来自于 retrieval。根据词源学，trie 的发明者 Edward Fredkin 把它读作 `/ˈtriː/`，不过，大部分人把它读作 `/ˈtraɪ/`。



- **基数树 (Radix Tree)** 也称为压缩前缀树，是对 Trie 的空间优化。它将 Trie 中只有一个子节点的路径节点合并成一个节点，节点上存储的是一段字符序列（而非单个字符），从而减少树的深度和节点数量，提高空间利用率。基数树的边可以表示多个字符，查找时按块比较，适合处理长字符串和有长公共前缀的集合。



因此，基数树可以看作是前缀树的一种压缩和优化版本，兼具 Trie 的前缀查找特性和更高的空间效率。

8 参考资料

- 图解Vllm V1系列5：调度器策略 (Scheduler) : <https://zhuanlan.zhihu.com/p/1908153627639551302>

- LLM Load Balancing at Scale: Consistent Hashing with Bounded Loads: <https://www.kubeai.org/blog/2025/02/26/llm-load-balancing-at-scale-chwbl/>
- SGLang Router for Data Parallelism: <https://docs.sglang.ai/router/router.html>
- SGLang v0.4: Zero-Overhead Batch Scheduler, Cache-Aware Load Balancer, Faster Structured Outputs: <https://lmsys.org/blog/2024-12-04-sglang-v0-4/>
- vLLM的prefix cache为何零开销: <https://zhuanlan.zhihu.com/p/1896927732027335111>
- Fast and Expressive LLM Inference with RadixAttention and SGLang: <https://lmsys.org/blog/2024-01-17-sglang/>
- EP05-vLLM源码讲解直播笔记-Prefix Caching: <https://kevincheung2259.github.io/2025/04/16/vLLM-EP05/>
- [Prefill优化][万字]🔥原理&图解vLLM Automatic Prefix Cache(RadixAttention): 首Token时延优化: <https://zhuanlan.zhihu.com/p/693556044>
- 图解Vllm V1系列6: KVCacheManager与PrefixCaching: <https://mp.weixin.qq.com/s/TaZjh-2gZlAEiFOjcSJHVw>
- 图解大模型计算加速系列: vLLM源码解析3, Prefix Caching: <https://mp.weixin.qq.com/s/bAY4OGqQIEeBaITlwxEuw>
- Prefix Cache Aware Proposal: <https://github.com/kubernetes-sigs/gateway-api-inference-extension/issues/498>
- AIBrix v0.3.0 发布: KVCache 多级卸载、前缀缓存、公平路由与基准测试工具: https://mp.weixin.qq.com/s/1_uUX7xMoQ6q7HFXrP2Bw
- 大模型推理加速与KV Cache (五) : Prefix Caching: <https://zhuanlan.zhihu.com/p/739669365>
- CoT系列-Self-Consistency(year 2022.Mar, Google): <https://zhuanlan.zhihu.com/p/609739922>
- PR [Experimental] Prefix Caching Support: <https://github.com/vllm-project/vllm/pull/1669>
- PR Add Automatic Prefix Caching: <https://github.com/vllm-project/vllm/pull/2762>
- SgLang代码细读-3.Cache: <https://www.cnblogs.com/sunstrikes/p/18891538>